PRODUCTION MLOPS PLATFORM

Implementation Checklist

A file-level, verifiable checklist for taking the NYC Airbnb pricing platform from its current foundation to a complete, demonstrated, production-shaped MLOps portfolio project. Each box describes a concrete deliverable or proof of completion.

CURRENT POSITION	WORKING CADENCE	REMAINING CHECKS
Foundation implemented	16 focused hours/week	57 verifiable items

Already present: feature engineering, Feast definitions, offline/online verification, the Airflow feature-build DAG, CatBoost training and MLflow logging, registry-alias loading, FastAPI serving, Docker, Helm, ArgoCD foundations, Terraform modules, validation CI, and unit tests. The checklist below covers the remaining work.

01 Data Validation & Training DAG

- [ ] Add training/validation.py: validate required columns, nulls, types, category values, coordinate bounds, non-negative counts, and target range; return a machine-readable report.
- [ ] Update pipelines/feature-build.py so validation runs after transformation and fails the DAG before Feast materialization when any blocking rule fails.
- [ ] Add pipelines/training.py with tasks for validation → point-in-time Feast retrieval → train → evaluate → log to MLflow → conditional registration.
- [ ] Reuse training/train.py inside the DAG; log dataset SHA-256, Git SHA, feature-service name, parameters, MAE/RMSE/R^2, model, and evaluation report.
- [ ] Add a simple baseline run and the CatBoost candidate to the same MLflow experiment so the UI provides a direct comparison.
- [ ] Define the registration gate in code: candidate must beat the configured MAE threshold and complete artifact-reload verification before registration.
- [ ] Add tests/test_validation.py and tests/test_training_dag.py, including one valid dataset and one dataset that is deliberately blocked.

02 Serving Observability & Model Refresh

- [ ] Add prometheus-client and expose GET /metrics from serving/app/api.py.
- [ ] Instrument request totals by endpoint/status, prediction latency histogram, prediction failures, readiness, loaded model version, and predicted-price histogram.
- [ ] Extend model_loader.py to resolve and retain the MLflow model version behind the configured alias, not only the model object.
- [ ] Add a safe refresh mechanism: periodically check the alias, atomically swap models after a successful load, and retain the current model if refresh fails.
- [ ] Extend tests/test_serving.py and tests/test_model_loader.py to verify metrics, version reporting, successful refresh, and failed-refresh fallback.
- [ ] Run a reproducible load test and record request rate plus p50/p95/p99 latency in docs/SERVING.md.

03 Drift Monitoring, Dashboards & Alerts

- [ ] Create monitoring/drift_check.py using Evidently; compare a configurable production window with the training reference dataset.
- [ ] Check numerical and categorical feature drift plus prediction drift; write JSON and HTML reports with timestamps and model-version metadata.
- [ ] Create a deliberately shifted test dataset and tests/test_drift_check.py; verify the normal sample passes and shifted features are flagged.
- [ ] Expose per-feature drift score, dataset drift state, drifted-feature count, and last-success timestamp to Prometheus.
- [ ] Add a scheduled Airflow monitoring DAG that runs the drift check and emits a retraining event only after the configured threshold is exceeded.
- [ ] Add three version-controlled Grafana JSON dashboards: infrastructure/service health, model/drift health, and Airbnb pricing business metrics.
- [ ] Add Prometheus rules for stale monitoring, feature drift, prediction drift, elevated error rate, and p95 latency.
- [ ] Add docs/runbooks/ entries describing impact, diagnosis, mitigation, rollback, and ownership for every alert.

04 Retraining, Approval, Promotion & Rollback

- [ ] Create retraining/retrain.py as an Airflow DAG triggered by the monitoring event, with the feature/data window recorded as run metadata.
- [ ] Evaluate challenger and current champion on the identical held-out dataset; require a configured minimum MAE improvement and non-regression guardrails for RMSE and key segments.
- [ ] If gates fail, retain the champion and record the rejection reason; if gates pass, register the challenger without moving the production alias.
- [ ] Add a human approval task before production promotion using an Airflow-supported approval mechanism and an audit trail.
- [ ] On approval, save the previous version as the rollback target and atomically move the MLflow production/champion alias.
- [ ] Run post-promotion smoke and performance checks; automatically restore the previous alias when health, latency, or model checks fail.
- [ ] Add tests for rejected challenger, approved promotion, missing approval, failed deployment check, and successful rollback.
- [ ] Verify serving observes the alias change and loads the promoted model without a code or image change.

05 Canary Delivery, CI/CD & GitOps

- [ ] Choose and document one canary mechanism (Argo Rollouts is the most direct fit with the existing ArgoCD setup).
- [ ] Extend serving/helm with stable/canary services and rollout analysis based on errors and latency; begin with a small traffic weight and promote or abort automatically.
- [ ] Expand .github/workflows/validate.yml to lint all Python, run all tests, validate every Terraform environment, lint Helm, and validate rendered manifests.
- [ ] Add an image workflow that builds, scans, tags with immutable Git SHA, authenticates using GitHub OIDC, and pushes serving/training images to ECR.
- [ ] Add a GitOps update step that changes the image tag in environment values; allow ArgoCD to reconcile instead of running kubectl from CI.
- [ ] Protect production with a GitHub Environment approval and keep registry promotion approval separately auditable.
- [ ] Verify a failed canary aborts without affecting stable traffic and a passing canary reaches 100%.

06 Cloud Infrastructure — Final Pass

- [ ] Wire the existing S3, RDS, and Redis modules into the environment roots; use smaller dev sizing and production-appropriate retention/HA settings.
- [ ] Add least-privilege Pod Identity roles and security-group paths for MLflow, Airflow training, Feast/Redis, serving artifact reads, and monitoring.
- [ ] Deploy MLflow in EKS with RDS/Postgres metadata and S3 artifacts; add probes, resource requests/limits, secrets, and persistent configuration.
- [ ] Deploy Airflow with DAG delivery, executor configuration, logs, secrets, and access to training nodes and required AWS services.
- [ ] Deploy Prometheus, Grafana, Alertmanager, and the required ServiceMonitor/PodMonitor resources.
- [ ] Apply dev first; verify nodes, pods, DNS, TLS, database connectivity, Redis connectivity, S3 writes, and registry model loading before staging/prod.
- [ ] Record actual cost-relevant resource sizes, capture evidence, then run the documented teardown to stop unnecessary charges.

07 End-to-End Verification

- [ ] Run the feature DAG and prove one point-in-time offline value matches the materialized Redis online value.
- [ ] Run the training DAG and show validation, MLflow logging, evaluation, and both passing and failing registration gates.
- [ ] Send a valid /predict request and capture model name/version plus the corresponding Prometheus request and latency metrics.
- [ ] Inject dramatic drift; capture the Evidently report, Prometheus drift metric, alert, and retraining-DAG trigger.
- [ ] Show challenger comparison, human approval, new registry version, alias movement, and automatic serving refresh.
- [ ] Force a post-promotion failure and prove the alias and serving model return to the previous version.
- [ ] Run the canary and load tests; retain command output and measured throughput/p50/p95/p99 latency.

08 Documentation & Portfolio Evidence

- [ ] Add docs/ARCHITECTURE.md with the data, control, deployment, observability, security, and rollback flows plus a rendered architecture diagram.
- [ ] Expand README.md with problem/metric, prerequisites, fresh-clone quick start, lifecycle, model results, API, monitoring, costs, trade-offs, and cleanup.
- [ ] Add dataset source, license/usage terms, data dictionary, versioning method, and reproducibility instructions.
- [ ] Add docs/PROMOTION_POLICY.md containing exact thresholds, approval responsibility, rollback conditions, and audit fields.
- [ ] Capture labeled screenshots: cluster, Feast materialization, MLflow runs, registry alias, prediction, drift, Grafana, registration gates, retraining, canary, and rollback.
- [ ] Record a short demo showing drift → alert → retraining → comparison → approval → promotion → serving refresh → rollback.
- [ ] Perform a clean-clone rehearsal using only documented commands; fix every missing prerequisite or unstated manual step.

DEFINITION OF DONE

Drift is detected → retraining is triggered → the challenger is evaluated → human approval is recorded → the registry alias is promoted → serving loads the new version → rollback is proven.